



HACKFEST 2026

Problem Statement Documentation & Technical Submission

ZENITH — PS17 · Domain 2, Cybersecurity & Blockchain

Team ID	KH050
Team Name	MetaWiz
Team Leader Name	Indra Kurkute
Problem Statement No.	PS17
Problem Statement Name	Attack Path & Identity Privilege Graph Analyzer
Date of Submission	11th September 2026



SECTION 1 — TEAM DETAILS

Team ID

KH050

Team Name

MetaWiz

Team Leader Name

Indra Kurkute

Team Members Name(s)

List the names of all other team members.

Indra Kurkute (Team Leader), Prathamesh Andhare, Ashish Lawande, Aalok Aage

Problem Statement Number

PS17

Problem Statement Name

Attack Path & Identity Privilege Graph Analyzer

SECTION 2 — UNDERSTANDING OF THE PROBLEM

Problem Statement

Explain the problem your team is solving, in your own words.

Every organization that runs Active Directory or a cloud identity system (Azure AD / Entra ID, AWS IAM) has hundreds of small permissions in place — who can log into which machine, who can reset whose password, who administers which server. Looked at one at a time, almost every one of these permissions looks perfectly safe and passes a standard security audit. The danger appears only when several ordinary, individually-compliant permissions are chained together: a low-privilege account can silently hop through three or four "safe" permissions and land on full Domain Admin access.

PS17 asks teams to build a tool that automatically discovers these hidden, multi-hop attack paths before a real attacker finds them first.

Problem Analysis / Background

What is the current situation or existing problem? Who is affected by it?

Today, most security teams audit permissions one setting at a time using checklists and vulnerability scanners. These tools catch a single misconfiguration well, but none of them reason about how permissions connect to one another. As a result, an organization can pass every individual compliance check and still have a silent, walkable path from a junior employee to its most powerful account. A realistic example: A junior helpdesk technician is allowed to RDP into a developer's workstation.

- That workstation happens to hold a cached login session for a backup service account.
- That service account has rights to reset the password of a delegated group.
- That group has administrator rights on the primary Domain Controller.

Individually, all four permissions above are "compliant." Chained together, they form a silent path from a low-privilege helpdesk account to full Domain Admin compromise — this is the exact pattern of lateral movement and privilege escalation seen in real breach investigations, and it affects essentially every mid-to-large organization running Active Directory or cloud IAM.

Proposed Solution

Explain your proposed solution and how it addresses the selected Problem Statement.

ZENITH models an organization's identities (users, groups, service accounts) and assets (machines) as a directed graph, storing every permission as a weighted edge inside a Neo4j graph database. Each weight reflects how easy or noisy that permission is to exploit in the real world — direct group membership is trivial and silent, while abusing a domain trust is loud and complex. Given any starting identity and target, ZENITH runs a Cypher graph query to mathematically find the lowest-cost, most realistic attack path, and scores its risk with a fully transparent formula instead of a black-box AI guess. On top of path-finding it adds:

- **Blast Radius** — everything an attacker could reach if one identity were compromised.
- **Remediation Simulation** — click any permission, simulate revoking it, and instantly see the attack path collapse or reroute.
- **Decoupled AI Briefing** — a plain-English executive summary generated only after the graph math has proven the path.

SECTION 3 — RESEARCH

Research Conducted

Research, references, datasets, papers, websites, APIs, documentation, or other resources used.

The team studied SpecterOps BloodHound, the industry-standard open-source tool used to map Active Directory attack paths, to understand how real-world identity graphs are modelled and queried. Every relationship type in ZENITH's data model was mapped to its corresponding MITRE ATT&CK technique ID, grounding the tool in an established threat framework. We also studied the Neo4j Cypher query language for weighted shortest-path traversal, and reviewed public identity & access management guidance for how organizations are expected to reason about layered privilege.

Existing Solutions / Similar Work

Mention existing solutions/projects related to your idea and how yours is different or improved.

The closest currently available tool is Blood Hound - that's also based on Neo4j modeling of AD objects as a graph and path-finding using Cypher. ZENITH is an extension of this, in four specific aspects:

Exploit-difficulty based weighting in cost, such that "cheapest" is actually representative of what attackers consider cheapest (read as noisy/difficult path in Bloodhound).

Unified graph that represents on-prem Active Directory in addition to cloud IAM and service account connections.

The ability to virtually repair a compromise in an interactive simulator-Blood Hound says that the path exists, Zeniths also suggests and lets the administrator test a remediation in simulation.

A solely decoupled AI narration layer-this one only explains an already identified path. It doesn't participate in vulnerability discovery itself, but rather explains detected risks;

References / Sources

Provide links to the important sources used during research.

- SpecterOps BloodHound — <https://bloodhound.specterops.io>
- MITRE ATT&CK Framework — <https://attack.mitre.org>
- MITRE ATT&CK — Privilege Escalation (TA0004) — <https://attack.mitre.org/tactics/TA0004/>
- Neo4j Graph Database Documentation — <https://neo4j.com/docs/>
- Neo4j Cypher Manual — <https://neo4j.com/docs/cypher-manual/current/>
- react-force-graph (visualization library) — <https://github.com/vasturiano/react-force-graph>
- OWASP (application & identity security reference) — <https://owasp.org>

SECTION 4 — TECHNICAL DOCUMENTATION

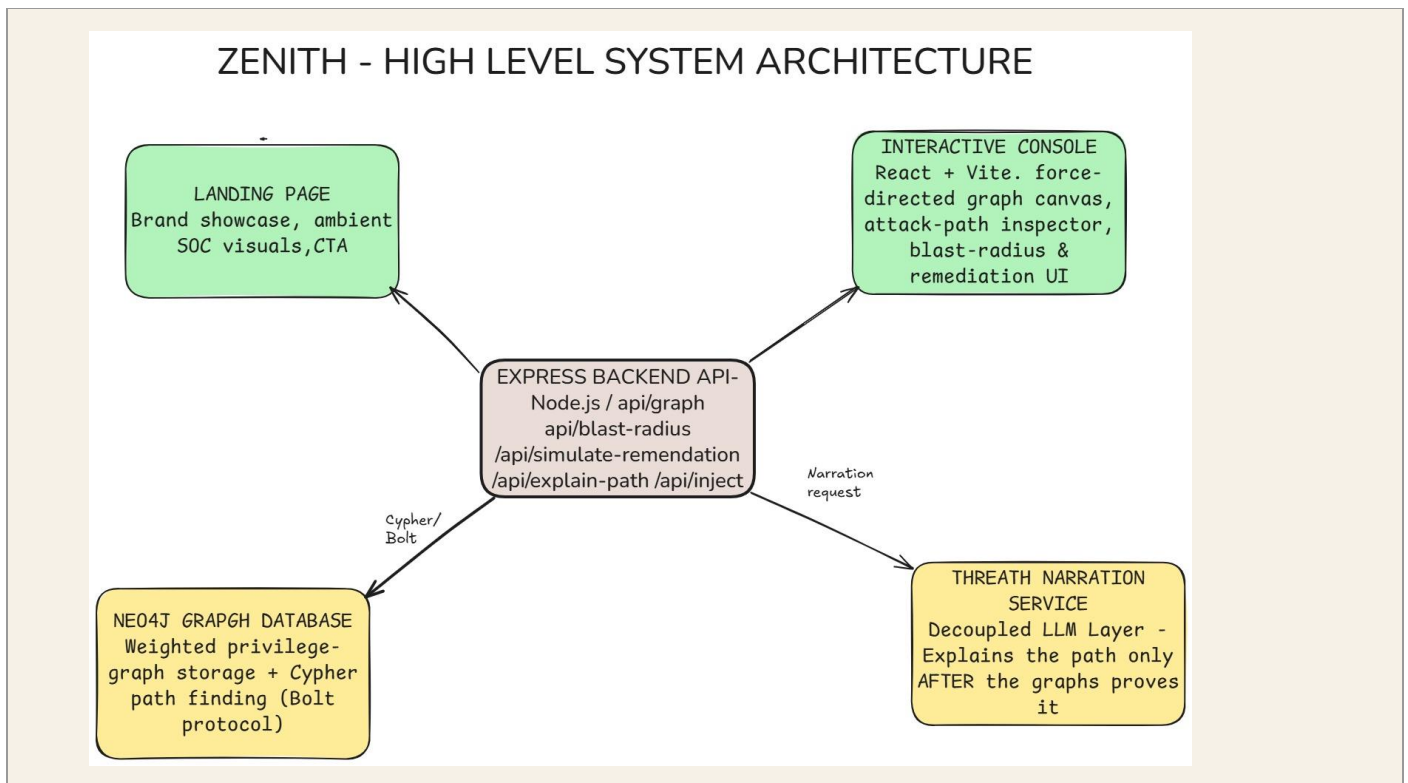
Technology Stack

Languages, frameworks, libraries, databases, APIs, cloud services, AI/ML models, etc.

Component	Technology	Why
Database	Neo4j + Cypher / GQL	Graph-native storage built for fast permission-path traversal
Backend API	Node.js + Express	Bolt-driver orchestration; exposes graph, path and simulation endpoints
Graph Visualization	react-force-graph-2d (Canvas)	Smooth, live rendering of hundreds of connected identities
Frontend Console	React 18 + Vite	Fast, modular interactive analyst console
Landing Page	HTML5 + CSS3	Brand front-door before entering the live console
Narration Layer	LLM API (decoupled)	Explains an already-proven path — never used to detect it

Technical Flow / System Architecture Diagram

Attach / paste your architecture diagram below.

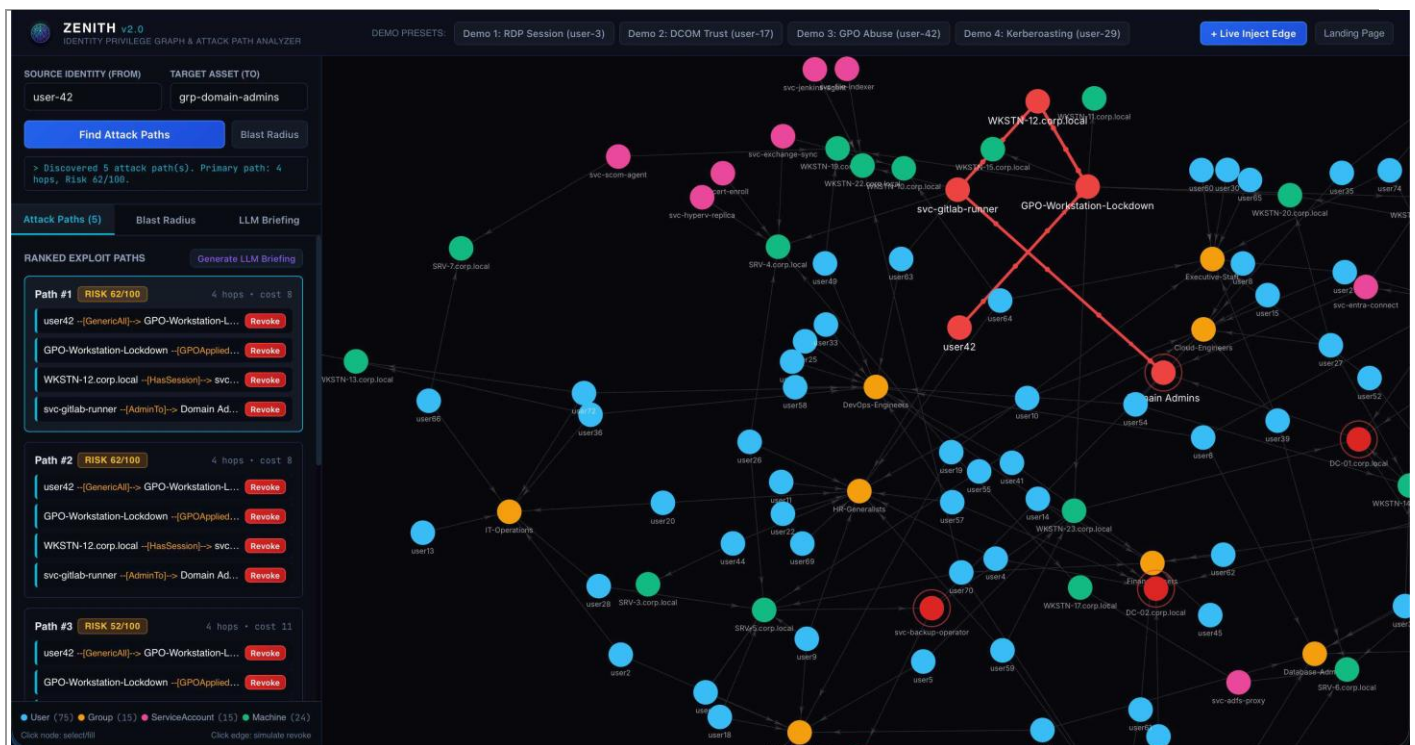


Working / Implementation Approach

Explain how your system works from input to output.

From input to output, ZENITH works in six steps: (1) a synthetic organization is generated and loaded into Neo4j as a directed graph, with every permission stored as a weighted edge; (2) the analyst picks a source identity and a target asset in the console — e.g. a low-privilege user and the Domain Admins group; (3) the API runs a Cypher query that returns the lowest-cost, most realistic path(s), each with its hop count and a transparent risk score; (4) the graph renders on a force-directed canvas with the discovered path highlighted; (5) the analyst can pull the Blast Radius of any node to see everything reachable if it were compromised; and (6) the analyst clicks any permission in the path to simulate revoking it — the backend re-checks the graph and reports whether the path is eliminated or reroutes.

Example — a live planted demo path: user-3 → (CanRDP) → HOST-5 → (HasSession, a cached credential) → svc-account-0 → (CanResetPasswordOf) → Domain Admins. Total exploit cost 9, Risk Score 58/100. Revoking just the "HasSession" permission between HOST-5 and svc-account-0 completely eliminates this path — the tool's core remediation moment, demoed live.



Live console — ranked attack paths from user-42 to grp-domain-admins, with per-hop risk scoring and one-click "Revoke" simulation.



Full organization graph loaded live from Neo4j (129 nodes, 197 weighted privilege edges).

AI/ML Details (if applicable)

Dataset, model, training approach, APIs/models used, evaluation, etc.

AI is used in exactly one place: the optional narration layer behind `/api/explain-path`. The graph algorithm always computes and mathematically proves the attack path first; the LLM is only called afterward to translate an already-proven result into a plain-English executive summary mapped to MITRE ATT&CK. The AI never detects, scores, or invents a vulnerability — it only narrates one the graph has already proven, keeping the tool fully explainable even if the AI/API layer is unavailable during judging.

GitHub Repository Link

Provide the team's public GitHub repository URL.

LINK :- <https://github.com/Conqueror17-AALOK/ZENITH->

SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

☐ I confirm that the solution submitted by our team is developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged.

Open-Source / External Resources Used

GitHub repositories, code snippets, templates, APIs, datasets, models, or other external resources used.

- Neo4j (graph database) and the Cypher query language.
- Node.js, Express, React 18, Vite — standard open-source frameworks used for the API and console.
- react-force-graph-2d — open-source force-directed graph rendering library.
- MITRE ATT&CK taxonomy — used as a conceptual reference for technique mapping, not as code.
- SpecterOps BloodHound — used only as an architectural / conceptual reference; no BloodHound code was copied or reused.

AI Tools Used

Mention AI tools used during development, if any, and briefly explain how they were used.

Claude (Anthropic) was used for architecture planning, technical write-up drafting, and generating this documentation file. All graph algorithms, Cypher queries, and the risk-scoring logic are original work designed and implemented by the team — AI assistance was used for planning and documentation support, not as a substitute for the underlying security logic. [Add any other AI-assisted coding tools used, e.g. GitHub Copilot / ChatGPT, so this stays accurate.]



SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- ☐ Technical Flow / Architecture Diagram - **DONE**
- ☐ Research / Reference Document (if applicable) - **DONE**
- ☐ Project Documentation / Technical Report - **DONE**
- ☐ GitHub Repository Link - **DONE**
- ☐ Screenshots of Working Prototype- **DONE**
- ☐ Dataset / Sample Data (if applicable) - **NA**
- ☐ Additional Diagrams (if any) - **NA**

Team Leader Signature: **Indra Kurkute**

Date: **11 / 09 / 2026**